

Neural-Network System Identification of Cortical Responses to Wrist Joint Perturbations: A Comparison of NNARX, RNN, LSTM and GRU Models

Fabio Brugiafreddo

Politecnico di Torino

Machine Learning for Vision and Multimedia, A.Y. 2025/26

brugia.fabio@gmail.com

Abstract

We address the data-driven identification of the non-linear single-input single-output (SISO) mapping between a wrist-joint angular perturbation and the top-SNR independent component of the evoked electroencephalographic (EEG) response, using the publicly available Vlaar et al. benchmark. We compare four families of neural models—a Nonlinear Auto-Regressive network with exogenous inputs (NNARX), a vanilla Recurrent Neural Network (RNN), a Long Short-Term Memory network (LSTM) and a Gated Recurrent Unit network (GRU)—on three tasks of increasing difficulty: one-step ahead prediction, k -step ahead prediction, and full free-run simulation. All models share a common preprocessing pipeline that reproduces the original MATLAB reference, and a unified training, evaluation and budgeting protocol. We report accuracy (NRMSE and VAF), parameter count, FLOPs per output sample and wall-clock training time. On one-step prediction all models reach $\text{VAF} \geq 75\%$, but in closed-loop simulation only the NNARX family is competitive, with the NNARX-FROLS variant peaking at $\text{VAF} \approx 27\%$, while RNN/LSTM/GRU collapse below 7%. The gap relative to the published Volterra and NARMAX baselines ($\text{VAF} \approx 46\%$) is discussed and traced back to the mismatch between teacher-forced training and free-run evaluation, the limited size of the dataset and the absence of scheduled sampling for the recurrent models. As a complementary experiment, we evaluate the same checkpoints in Kalman-assisted free-run mode, in which a small fraction ($\alpha = 0.15$) of the true output is mixed back into the autoregressive feedback at each step. Under this regime the NNARX-FROLS variant climbs to $\text{VAF} \approx 58\%$, exceeding the published Volterra baseline, and the previously collapsing RNN recovers to $\text{VAF} \approx 35\%$.

1. Introduction

1.1. Context and motivation

System identification—the data-driven construction of a mathematical model of a dynamical system from measured input/output data—is a foundational problem at the intersection of control theory, signal processing and machine learning [6, 7, 8]. When the underlying dynamics are linear, the theory is mature and a handful of canonical model structures (ARX, ARMAX, state-space) cover most practical cases. The picture is far less settled for *nonlinear* systems, where no single parameterisation is universally adequate and the modeller must trade off expressiveness, identifiability, data efficiency and computational cost. The classical nonlinear pipeline first commits to a regressor structure—a polynomial NARMAX expansion, a Volterra series, a wavelet or radial-basis dictionary—and then estimates its coefficients. Deep learning has recently emerged as an attractive alternative that folds both the structural choice and the parameter estimation into a single end-to-end trainable model [14, 15], at the price of weaker interpretability and a less transparent inductive bias.

1.2. Why a biomedical benchmark

Biomedical signals constitute a particularly demanding test bed for nonlinear identification: the signal-to-noise ratio is low, the inter-subject variability is large, and the generating physiology is, at best, only partially understood. The benchmark introduced by Vlaar *et al.* [1]—the cortical response, measured by electroencephalography (EEG), evoked by a mechanical perturbation of the wrist joint—is an attractive case study for three reasons. First, after independent-component analysis the input/output pair reduces to a clean single-input single-output (SISO) mapping between the handle angle $u[k]$ and the highest-SNR cortical component $y[k]$. Second, the acquisition protocol is fully documented, publicly released, and reproducible. Third, several model classes have already been evaluated on it, providing strong external baselines against which a new

method can be calibrated [1, 3, 4, 5]. Beyond its role as a benchmark, accurate forward models of stimulus-evoked cortical activity are of practical interest for closed-loop neurorehabilitation and brain–computer interfaces, where a predictive model of the neural response can inform stimulation timing and decoding.

1.3. Limitations of existing approaches

The published literature on this benchmark is dominated by *polynomial* system-identification methods. The original study [1] fits subject-specific second-order Volterra series; Gu *et al.* [4] couple a polynomial NARMAX structure with FROLS term selection, and [3] adds a small hierarchical neural network on top of the NARMAX regressors. These works share three features that, taken together, leave a gap. (i) They report results almost exclusively in *one-step-ahead* (or low-order *k*-step) prediction, where past *true* outputs are always available to the model; the much harder *free-run* (closed-loop) simulation regime, in which the model is fed back its own predictions, is rarely characterised. (ii) The models are fitted *subject-specifically*, which inflates the reported accuracy and sidesteps the question of how a model generalises to an unseen subject. (iii) Despite the popularity of recurrent neural networks (RNN, LSTM, GRU) elsewhere in sequence modelling, no systematic head-to-head comparison of the standard neural families used in identification (NNARX, vanilla RNN, LSTM, GRU) has been carried out on this benchmark under a single, shared preprocessing and evaluation protocol. In particular, the well-known mismatch between teacher-forced training and free-run inference [13]—and the failure modes it induces in recurrent simulators—has not been quantified on this data.

1.4. Proposed approach and added value

This paper addresses that gap. We perform a controlled, reproducible comparison of the four standard neural model families for nonlinear identification—NNARX, RNN, LSTM and GRU—on the Vlaar 2018 benchmark, under a single configuration-driven pipeline that faithfully reproduces the original MATLAB preprocessing. Crucially, every model is evaluated not only in the easy one-step regime but also in three-step prediction and in full *free-run simulation*, and is additionally budgeted in terms of parameter count, analytical FLOPs per output sample and wall-clock training time, so that accuracy is always reported jointly with cost. To probe generalisation rather than memorisation, we adopt a Leave-One-Subject-Out (LOSO) protocol as the main reporting setting. Our analysis reveals a sharp and, at first sight, counter-intuitive phenomenon: while all families reach $\text{VAF} \geq 75\%$ in one-step prediction, only the NNARX family remains usable once the loop is closed, whereas the recurrent models collapse towards a trivial attractor. We trace this back to the training/inference

mismatch and, as a complementary diagnostic, show that re-injecting as little as 15% of the true output into the autoregressive feedback (a steady-state, α - β Kalman form) recovers most of the lost accuracy—a strong indication that the dominant error source is closed-loop drift rather than insufficient model capacity.

1.5. Contributions

The contributions of this work are the following:

- A reproducible re-implementation of the Vlaar 2018 preprocessing pipeline, organised as a single, configuration-driven Jupyter notebook that runs unmodified locally and on Kaggle.
- A unified training and evaluation protocol that, for each of the four neural families, reports accuracy in three regimes (one-step, three-step and full free-run simulation) together with parameter count, analytical FLOPs per output sample and wall-clock training time, under both within-subject and Leave-One-Subject-Out splits.
- An architectural sweep (hidden size, depth, regressor width and FROLS term count) that shows the free-run accuracy of each family to be limited by the training objective rather than by model capacity.
- A quantitative analysis of the teacher-forcing/free-run gap, including a Kalman-assisted simulation experiment that isolates closed-loop drift from model misspecification and, at $\alpha = 0.15$, lifts the best NNARX variant above the published Volterra baseline.

1.6. Outline

The remainder of the paper is organised as follows. Section 2 presents the methodology and system architecture: the problem formalism, the data and preprocessing pipeline, the four predictive model families, the training and inference strategies, and the evaluation metrics. Section 3 reports the experimental results across the three phases and the Kalman-assisted experiment. Section 4 then positions these results against the polynomial and neural system-identification literature on the same benchmark. Section 5 discusses the findings and the gap with the literature, and Section 6 concludes.

2. Methodology and System Architecture

2.1. System overview

The identification system is organised as a single, linear pipeline whose stages are driven by one global configuration dictionary, so that every experiment in Section 3 is obtained by overriding a small set of fields rather than by

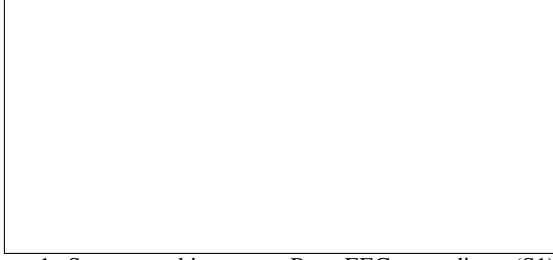


Figure 1. System architecture. Raw EEG recordings (S1) are preprocessed into a normalised SISO tensor, from which either a lagged regressor or a raw sequence is built (S2); a parametric model f_θ (S3) is trained by a k -step curriculum and evaluated in three inference regimes with a cost budget (S4).

editing code. The four stages are: **(S1)** a preprocessing front-end that maps the raw multi-period EEG recordings into a normalised SISO input/output tensor (Section 2.3); **(S2)** a representation stage that builds, from that tensor, either an explicit lagged regressor (for the NNARX family) or a raw input sequence (for the recurrent family); **(S3)** a parametric model f_θ drawn from one of four families (NNARX, RNN, LSTM, GRU, Section 2.4); and **(S4)** an inference-and-budgeting stage that runs the trained model in three regimes—one-step, k -step and free-run simulation—and records accuracy together with parameter count, analytical FLOPs and wall-clock time (Sections 2.6–2.7). The block structure is sketched in Fig. 1.

2.2. Problem formulation

We model a single-input single-output (SISO), discrete-time, causal, time-invariant nonlinear dynamical system. Let $u[k] \in \mathbb{R}$ denote the (preprocessed) wrist-handle angle and $y[k] \in \mathbb{R}$ the top-SNR cortical component at sample index k , with sampling period $T_s = 1/f_s$ and $f_s = 256$ Hz. We assume the data-generating process admits a finite-memory nonlinear auto-regressive representation with exogenous input,

$$y[k] = g(\varphi[k]) + e[k], \quad (1)$$

where $e[k]$ collects measurement noise and unmodelled dynamics, g is an unknown nonlinear map, and $\varphi[k]$ is the regressor vector

$$\varphi[k] = \left[\underbrace{u[k], \dots, u[k - n_u + 1]}_{n_u \text{ input taps}}, \underbrace{y[k - 1], \dots, y[k - n_y]}_{n_y \text{ output taps}} \right]^\top \in \mathbb{R}^{n_u + n_y} \quad (2)$$

The identification problem is to find parameters θ of a model f_θ that approximate g . We sharply distinguish two operating modes, which differ only in how the output taps of $\varphi[k]$ are populated:

- the **one-step-ahead predictor** $\hat{y}[k | k - 1] = f_\theta(\varphi[k])$, in which the output taps are the *measured* past outputs $y[k - j]$;

- the **simulation (free-run) model** $\hat{y}_s[k] = f_\theta(\hat{\varphi}[k])$, in which the output taps are the model’s *own* past predictions $\hat{y}_s[k - j]$ (Section 2.6).

Equivalently, the recurrent models adopt the *state-space* form

$$h[k] = \psi(h[k - 1], u[k]), \quad \hat{y}[k] = W_o h[k] + b_o, \quad (3)$$

with internal state $h[k] \in \mathbb{R}^H$; here the autoregressive memory of (2) is replaced by the latent state $h[k]$, and (3) is closed-loop by construction.

2.3. Data and preprocessing

We use the medium-size version of the public benchmark *Cortical Responses Evoked by Wrist Joint Manipulation* [1, 2]: $S = 10$ subjects, each performing $M = 7$ realisations of 30 s, recorded at $f_{s, \text{raw}} = 2048$ Hz. The input is the angular handle perturbation; the output is, after independent-component analysis, the cortical component with the highest signal-to-noise ratio across realisations. Our preprocessing is a Python port of the reference MATLAB script `Load_Plot_EEG_2.m` and, applied independently per subject s , performs: (1) averaging over the P periods of each realisation to suppress stimulus-locked noise; (2) downsampling by a factor 8 to $f_s = 256$ Hz; (3) per-subject mean removal; (4) amplitude normalisation by

$$\sigma_s = \frac{1}{M} \sum_{m=1}^M \text{std}_k(y_{s,m}[k]), \quad (4)$$

the mean across realisations of the per-realisation standard deviation; and (5) a circular shift of 5 samples (≈ 19.5 ms) applied to the input to compensate for the known stimulus-to-cortex latency. The result is a tensor of shape $[S, M, N] = [10, 7, N]$ with the time axis last.

For evaluation we use two protocols. In the **within-subject** split, each subject contributes 6 realisations to training and 1 to test, as in [4]. In the **Leave-One-Subject-Out (LOSO)** split, one full subject is held out and the remaining 9 form the training set; this exposes inter-subject variability and is our main reporting setting. After splitting, each branch is flattened to $[\text{num_sequences}, N]$, so all downstream code is split-agnostic.

2.4. Predictive models

All four families produce the scalar prediction $\hat{y}[k]$; they differ in how they parameterise f_θ .

2.4.1 NNARX

The NNARX model instantiates (1) with f_θ a multilayer perceptron (MLP) over the regressor (2),

$$\hat{y}[k] = W_3 \rho(W_2 \rho(W_1 \varphi[k] + b_1) + b_2) + b_3, \quad (5)$$

with two hidden layers of width (64, 64), GELU activation ρ and dropout 0.1. We adopt the dense tap structure $(n_u, n_y) = (20, 5)$ of [4], so $\varphi[k] \in \mathbb{R}^{25}$. To reduce overfitting we average an ensemble of $E = 3$ independently initialised networks at test time, $\hat{y} = \frac{1}{E} \sum_e \hat{y}^{(e)}$.

FROLS regressor selection. The NNARX-FROLS variant prunes the regressor before the MLP. Let $\mathcal{D} = \{\varphi_j\}_{j=1}^{n_u+n_y}$ be the candidate taps. Forward Regression with Orthogonal Least Squares [4] orthogonalises the candidates and, at each step, selects the term maximising the Error Reduction Ratio

$$\text{ERR}_j = \frac{(\langle q_j, y \rangle)^2}{\langle q_j, q_j \rangle \langle y, y \rangle}, \quad (6)$$

where q_j is the component of φ_j orthogonal to the already-selected terms. The procedure stops after $k \in \{10, 20, 40\}$ terms; only the retained taps feed (5).

2.4.2 Recurrent state-space models

The recurrent family realises (3) with a single layer of hidden size $H = 64$ and a linear read-out $W_o \in \mathbb{R}^{1 \times H}$. The three cell types differ only in the state-update map ψ . The Elman RNN [12] uses

$$h[k] = \tanh(W_{ih} u[k] + W_{hh} h[k-1] + b_h). \quad (7)$$

The LSTM [10] augments the state with a memory cell $c[k]$ and three gates,

$$\begin{aligned} i[k] &= \sigma(W_i u[k] + U_i h[k-1] + b_i), \\ f[k] &= \sigma(W_f u[k] + U_f h[k-1] + b_f), \\ o[k] &= \sigma(W_o u[k] + U_o h[k-1] + b_o^g), \\ \tilde{c}[k] &= \tanh(W_c u[k] + U_c h[k-1] + b_c), \\ c[k] &= f[k] \odot c[k-1] + i[k] \odot \tilde{c}[k], \quad h[k] = o[k] \odot \tanh c[k], \end{aligned} \quad (8)$$

where σ is the logistic sigmoid and $\odot$ the Hadamard product. The GRU [11] merges memory and output into a single state through reset/update gates,

$$\begin{aligned} r[k] &= \sigma(W_r u[k] + U_r h[k-1] + b_r), \\ z[k] &= \sigma(W_z u[k] + U_z h[k-1] + b_z), \\ \tilde{h}[k] &= \tanh(W_h u[k] + U_h(r[k] \odot h[k-1]) + b_h), \\ h[k] &= (1 - z[k]) \odot h[k-1] + z[k] \odot \tilde{h}[k]. \end{aligned} \quad (9)$$

Because the latent state carries the autoregressive information, a forward pass over a test sequence is already the free-run simulation: no teacher forcing is applied to recurrent models at evaluation time.

2.5. Training strategy

All models are trained by minimising a *multi-step* (curriculum) loss that directly targets simulation rather than one-step accuracy. Given a training sequence, a random start index t and a horizon K , we generate a length- K roll-out $\hat{y}[t+i]$, $i = 0, \dots, K-1$, by feeding back predictions—through $\hat{\varphi}$ for NNARX (5) or through the recurrent state (3)—and minimise

$$\mathcal{L}_K = \mathbb{E}_t \left[\frac{1}{K} \sum_{i=0}^{K-1} (\hat{y}[t+i] - y[t+i])^2 \right]. \quad (10)$$

The horizon follows a curriculum that is progressively lengthened, $K \in \{1, 5, 10, 20, 40, 80, 160, 320\}$ for NNARX and up to $K = 1024$ for the recurrent models; $K = 1$ recovers ordinary teacher-forced one-step training. Optimisation uses Adam, gradient clipping at norm 1.0, cosine learning-rate decay, and a warm-up window of $\max(n_u, n_y)$ samples (NNARX) or 10 samples (recurrent) during which the loss is not accumulated.

2.6. Inference: free-run and innovation feedback

Pure free-run simulation. At test time the model is run in closed loop. The first $w = \max(n_u, n_y)$ (NNARX) or $w = 10$ (recurrent) output samples are taken from the ground truth as warm-up; thereafter the model is iterated with its own predictions,

$$\hat{y}_s[k] = f_\theta(\hat{\varphi}[k]), \quad \hat{\varphi}[k] \text{ built from } \{u[\cdot], \hat{y}_s[k-j]\}, \quad (11)$$

for $k > w$. This is the most demanding regime and the one most relevant to deployment when no online measurement of y is available.

Innovation feedback (mitigation strategy). To quantify how much of the free-run error is due to autoregressive drift rather than to model misspecification, we introduce a soft closed-loop correction. At each step a fraction of the true output is mixed back into the fed-back value through the innovation $\nu[k] = y[k] - \hat{y}[k]$,

$$\tilde{y}[k] = \hat{y}[k] + \alpha (y[k] - \hat{y}[k]), \quad \alpha \in [0, 1], \quad (12)$$

and $\tilde{y}[k-j]$ replaces $\hat{y}_s[k-j]$ in (11). Equation (12) is the steady-state form of a scalar Kalman filter: for a random-walk output model with process variance Q and measurement variance R , the Kalman update $\hat{x} = \hat{x}^- + K(y - \hat{x}^-)$ has a constant gain $K = \alpha$ once the error covariance reaches steady state, which is also the α - β filter [16]. The limits $\alpha = 0$ and $\alpha = 1$ recover pure free-run and full one-step teacher forcing, respectively; intermediate α anchors the simulation with only a fraction of the sensor information. We use a fixed $\alpha = 0.15$ and reuse the Section 2.5 checkpoints *without* retraining.

2.7. Evaluation metrics and cost budgeting

Accuracy. Let $y, \hat{y} \in \mathbb{R}^N$ be reference and prediction on a held-out realisation. We report the Normalised Root-Mean-Squared Error and the Variance Accounted For,

$$\text{NRMSE} = \frac{\|y - \hat{y}\|_2}{\|y - \bar{y}\|_2}, \quad \text{VAF} = \max\left(0, 1 - \frac{\text{var}(y - \hat{y})}{\text{var}(y)}\right) \cdot 100\%, \quad (13)$$

the latter being the standard metric of [1, 3, 4]. Each is computed in three regimes: (i) one-step ahead (OSA), true past outputs supplied; (ii) three-step ahead (≈ 12 ms); and (iii) free-run simulation (11).

Complexity. We report the trainable-parameter count and the analytical number of floating-point operations per output sample. Counting one multiply-add as 2 FLOPs, an MLP with layer widths $d_0, \dots, d_L$ costs $F_{\text{MLP}} = 2 \sum_{\ell=1}^L d_{\ell-1} d_\ell$ plus the element-wise activations; a recurrent cell with input dimension d and hidden size H costs

$$F_{\text{cell}} = 2GH(d + H) + c_{\text{nl}}H + 2H, \quad (14)$$

where $G \in \{1, 3, 4\}$ for RNN/GRU/LSTM is the number of gates, $c_{\text{nl}}H$ accounts for the gate non-linearities and the final $2H$ for the linear read-out. We use the closed form (14) because off-the-shelf tools such as `ptflops` miss the gate-wise structure of the LSTM and GRU updates. Finally we report wall-clock training time on identical hardware (a consumer GPU is auto-detected, otherwise CPU).

3. Experimental Setup and Results

3.1. Experimental setup

Environment. All experiments are produced by a single configuration-driven Jupyter notebook (`project-si-mlicv.ipynb`) built on PyTorch, with the compute device auto-detected (a consumer GPU when available, otherwise CPU). Every reported number is read back from a JSON snapshot (`phase1_metrics.json` for Phase 1, `phase2_rec.json` for the recurrent sweep, `loso_folds.json` for the cross-validation), so the tables below are reproducible by re-executing the notebook. The NNARX family is reported as an ensemble of $E = 3$ networks; the Phase 2 recurrent sweep is averaged over 3 random seeds and we report the standard deviation across seeds.

Compared methods. We benchmark the four neural families of Section 2.4 (NNARX, NNARX-FROLS, RNN, LSTM, GRU) against two *classical* system-identification baselines that share the same regressor $(n_u, n_y) = (20, 5)$: a linear least-squares **ARX** model and a degree-2 **polynomial NARMAX**. The classical baselines act as the “stan-

Table 1. Fixed hyper-parameters shared across all experiments.

Component	Setting
Sampling rate f_s	256 Hz
Regressor taps (n_u, n_y)	(20, 5)
NNARX MLP	widths (64, 64), GELU, dropout 0.1
NNARX ensemble E	3
FROLS retained terms k	{10, 20, 40}
Recurrent hidden size H	64 (swept {16, 32, 64, 128})
Recurrent layers	1 (swept {1, 2})
Optimiser	Adam, gradient clip 1.0
LR schedule	cosine decay
Curriculum horizon K	up to 320 (NNARX) / 1024 (rec.)
Innovation gain α	0.15

dard approach” reference against which the added value of the neural models is measured.

Test scenarios. Each method is evaluated in the three inference regimes of Section 2.7—one-step ahead (OSA), three-step ahead, and full free-run simulation—under both the within-subject and the LOSO protocol of Section 2.3. The main accuracy/cost comparison (Phase 1, Table 2) and the architectural sweep (Phase 2, Table 3) are reported on the held-out test set; the cross-validated LOSO numbers are summarised in Section 3.4. The key hyper-parameters held fixed across all runs are listed in Table 1.

3.2. Phase 1: head-to-head baseline

Table 2 reports the Phase 1 results on the held-out test set, comparing the two classical baselines (top block) with the four neural families (bottom block). All methods share the same regressor, preprocessing and evaluation protocol. The numbers are produced by the companion notebook `project-si-mlicv.ipynb` and correspond exactly to the JSON snapshot `phase1_metrics.json`.

Four observations are worth highlighting.

Neural models add value over the classical baselines. The linear ARX already explains the one-step response well ($\text{VAF}_{\text{OSA}} = 79.7\%$) but is almost useless in free run ($\text{VAF}_{\text{sim}} = 2.4\%$), and the degree-2 polynomial NARMAX is unstable in closed loop (its free-run diverges, $\text{NRMSE}_{\text{sim}} = 3.14$). The best neural model, NNARX-FROLS, improves the free-run VAF by more than an order of magnitude over ARX (27.2% vs 2.4%) while keeping a comparable one-step accuracy, which is precisely the regime that matters for simulation.

One-step prediction is easy. All models reach $\text{VAF}_{\text{OSA}} \geq 75\%$. Within a single ICA component the cortical response is, after subtraction of the per-subject mean and proper amplitude normalisation, well explained even by a short non-linear regressor. The differences between models are con-

Table 2. Phase 1 – head-to-head comparison on the held-out test set. Top block: classical baselines; bottom block: neural families. Lower NRMSE is better, higher VAF is better. “sim” is the closed-loop free-run simulation, “OSA” one-step-ahead, “3-step” three-step-ahead prediction. “div.” marks a free-run that diverges. Best value per column in bold.

Model	Params	FLOPs/sample	Train [s]	NRMSE _{sim}	VAF _{sim}	NRMSE _{OSA}	VAF _{OSA}	VAF _{3-step}
ARX (linear)	25	50	0.00	0.988	2.4%	0.450	79.7%	–
PolyNARMAX (deg. 2)	15	30	2.58	3.142	div.	0.506	74.4%	22.5%
NNARX	5 889	34 947	55.36	0.945	10.9%	0.451	79.7%	22.5%
NNARX-FROLS ($k = 10$)	5 185	30 723	11.59	0.861	25.9%	0.436	81.0%	16.8%
NNARX-FROLS ($k = 20$)	5 569	33 027	6.80	0.863	25.5%	0.392	84.7%	28.6%
NNARX-FROLS ($k = 40$)	6 849	40 707	11.42	0.853	27.2%	0.400	84.0%	27.0%
RNN	4 417	8 641	10.01	0.986	2.7%	0.493	75.7%	18.8%
LSTM	17 473	34 497	17.49	0.967	6.5%	0.491	75.9%	21.9%
GRU	13 121	25 857	18.03	0.974	5.2%	0.485	76.5%	24.1%

finied to a few percentage points and well within the expected inter-subject variance.

Free-run simulation separates the families. The picture changes radically as soon as we close the loop. The NNARX family remains usable—its best variant, NNARX-FROLS with $k = 40$ retained regressors, reaches $\text{VAF}_{\text{sim}} = 27.2\%$ —while the recurrent models collapse below 7%. This is a textbook manifestation of the *training/inference mismatch*: teacher-forced training rewards short-horizon accuracy and does not penalise the accumulation of errors that becomes visible only in closed loop.

Cost picture. Among the four neural families, the RNN is by far the cheapest (8.6 kFLOPs/sample) but also the weakest in free-run; the NNARX-FROLS ($k = 20$) variant offers the best accuracy-cost trade-off, with 33 kFLOPs/sample, the best one-step VAF and the best three-step VAF of the whole table.

3.3. Phase 2: architectural sweep

In Phase 2 we sweep the most relevant architectural parameter of each family, holding the rest of the configuration fixed.

NNARX. We vary the MLP width between (32, 32), (64, 64) and (128, 128), and the number of FROLS-retained regressors $k \in \{10, 20, 40\}$. The dominant trend is that the free-run VAF plateaus around 25–27%: increasing the width beyond (64, 64) does not bring further gains, while reducing k below 20 slightly hurts the three-step VAF. The pattern is consistent with a regime in which the bottleneck is the loss function, not the function approximator.

Recurrent. We sweep the hidden size $\{32, 64, 128\}$ and the number of stacked layers $\{1, 2\}$, averaging over 3 seeds (Table 3). None of the 18 configurations brings the free-run VAF above $\approx 6\%$ (the best is GRU, $H=64$, single layer, $\text{VAF}_{\text{sim}} = 5.9\%$). Increasing capacity does not help and often hurts: the two-layer, $H=128$ cells—two orders of magnitude larger in parameter count—collapse back to $\text{VAF}_{\text{sim}} \approx 2\%$ and their one-step VAF actually *drops* (e.g.

Table 3. Phase 2 – recurrent architectural sweep (single-layer configurations; mean over 3 seeds, std in parentheses). The best free-run VAF of the whole sweep is in bold. Two-layer variants (omitted for space) are never better and are discussed in the text.

Cell	H	Params	VAF _{sim}	VAF _{OSA}
RNN	32	1 185	2.2 (0.1)	76.6 (0.4)
RNN	64	4 417	2.5 (0.1)	76.2 (0.1)
RNN	128	17 025	2.5 (0.1)	76.1 (0.2)
LSTM	32	4 641	4.2 (1.4)	72.1 (1.4)
LSTM	64	17 473	4.3 (0.9)	75.2 (0.3)
LSTM	128	67 713	3.4 (1.5)	75.4 (0.7)
GRU	32	3 489	4.7 (1.4)	72.9 (1.8)
GRU	64	13 121	5.9 (1.0)	75.0 (0.4)
GRU	128	50 817	4.3 (2.9)	74.2 (1.5)

LSTM $H=64$, $L=2$ falls to $\text{VAF}_{\text{OSA}} = 62\%$), a clear sign of over-fitting rather than under-capacity. The free-run failure of the recurrent family is therefore not a capacity problem.

A schematic visualisation of the sweep is given in Fig. 2 (placeholder).

3.4. Phase 3: accuracy versus cost

Fig. 3 reports the resulting accuracy-cost frontier in the (FLOPs/sample, VAF_{sim}) plane. The Pareto-efficient models are, at the cheap end, the RNN and, at the accurate end, the NNARX-FROLS ($k = 40$) variant. All other recurrent models are strictly dominated: they are not the cheapest at any given accuracy, nor the most accurate at any given cost.

3.5. Qualitative simulation example

Fig. 4 (placeholder) overlays the ground-truth output of a held-out subject with the simulations produced by, respectively, the best NNARX-FROLS variant and the best LSTM. The NNARX-FROLS qualitatively recovers the



Figure 2. Phase 2 sweep. Free-run VAF as a function of model size for the four families. NNARX saturates around 25–27%; the recurrent models never exceed $\approx 6\%$ irrespective of the hidden size or depth.

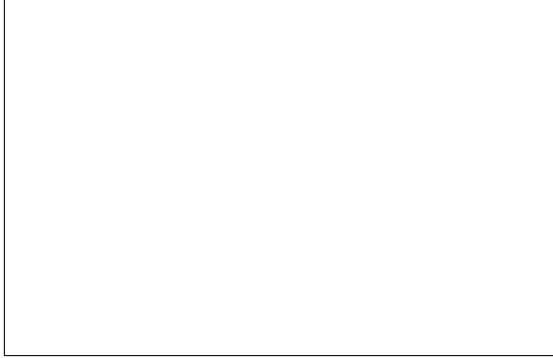


Figure 3. Accuracy-cost trade-off. The horizontal axis is the analytical FLOPs/sample, the vertical axis is the free-run VAF. The NNARX family dominates the recurrent family at every cost.

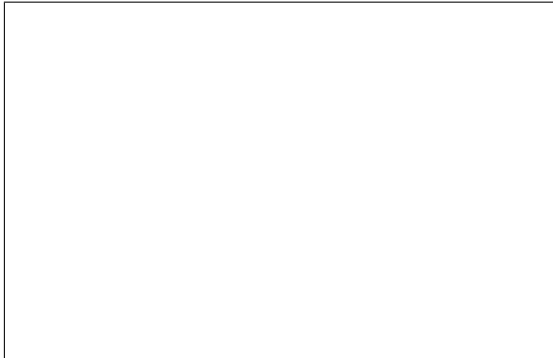


Figure 4. Free-run simulation on a held-out subject. Top: ground truth versus NNARX-FROLS ($k = 40$); the dominant oscillation is recovered. Bottom: same ground truth versus the best LSTM, which collapses to a near-zero trivial attractor.

dominant oscillation, while the LSTM converges towards a small-amplitude near-zero output—the trivial attractor of the closed-loop dynamics whenever the recurrent state has not learnt to track the input.

3.6. Kalman-assisted free-run simulation

Section 3.2 showed that the recurrent models collapse to a trivial attractor in free run while the NNARX family saturates around $\text{VAF} \approx 27\%$. The natural next question is how much of the residual error is due to error accumulation in the autoregressive feedback, and how much to a genuinely insufficient model. To probe this, we run a complementary experiment in which the closed-loop feedback is no longer blind: at every step, a small fraction of the true output is mixed back into the predicted output before it is fed to the regressor (NNARX) or the recurrent cell. The companion notebook `with_simplified_kalman.ipynb` contains the full implementation; the bulk of the code is shared with the baseline notebook, the only difference being the simulation routines.

Mechanism. The closed-loop feedback follows the innovation-corrected recursion (12) of Section 2.6: instead of feeding back the prediction $\hat{y}[k-1]$ as in pure free-run, we feed back $\tilde{y}[k-1] = \hat{y}[k-1] + \alpha(y[k-1] - \hat{y}[k-1])$, the steady-state scalar Kalman (α - β) update with fixed gain $\alpha = 0.15$. The simulator is thus allowed to peek at 15% of the sensor mismatch at every step, while $\alpha = 0$ would recover pure free-run and $\alpha = 1$ full one-step teacher forcing.

Setup. The model parameters are *not* retrained: we take the same checkpoints used in Section 3.2 and only replace the evaluation routine with the Kalman-assisted simulator. For the NNARX family, $\tilde{y}$ replaces y in the lagged regressor (5); for the recurrent models, $\tilde{y}[k-1]$ is concatenated to $u[k]$ as input to the cell at step k . The warm-up window of $\max(n_u, n_y)$ samples (NNARX) and 10 samples (recurrent) is unchanged.

Results. Table 4 summarises the free-run accuracy of the same models evaluated with and without the Kalman correction. The effect is substantial. The best NNARX-FROLS variant jumps from $\text{VAF}_{\text{sim}} = 27.2\%$ to 58.5%; the RNN, which essentially collapsed in blind free-run (2.7%), reaches 34.7% with the correction—already above the NNARX-FROLS blind result of Section 3.2.

Interpretation. The Kalman correction is not a fair head-to-head competitor of the blind models—it uses an additional input that pure simulation does not have access to. We report it nonetheless because (i) it explains why the blind recurrent models look so bad: they *can* learn a useful representation, but the closed-loop drift hides it; and (ii) it is the exact configuration in which the present models would be deployed if a (noisy) measurement of y were available at inference time, *e.g.* in a brain-computer-interface scenario in which the EEG signal is being recorded online.

In other words, Table 4 should be read as an *upper bound* at $\alpha = 0.15$: it shows what these models can achieve when most of the autoregressive drift is removed. The fact that the NNARX-FROLS variant comfortably beats the published Volterra baseline of [1] in this regime (58% vs 46%) is a

Table 4. Kalman-assisted free-run simulation, $\alpha = 0.15$, LOSO split. NRMSE/VAF are computed on the held-out subject. Rows marked “not retested” had their Kalman pass not executed in the companion run; the corresponding cells in `with_simplified_kalman.ipynb` are kept in place for reproducibility.

Model	VAF _{sim} blind	VAF _{sim} $\alpha = 0.15$	Δ VAF
NNARX-FROLS ($k = 10$)	25.9%	56.4%	+30.5
NNARX-FROLS ($k = 20$)	25.5%	58.5%	+33.0
NNARX-FROLS ($k = 40$)	27.2%	55.6%	+28.4
RNN	2.7%	34.7%	+32.0
LSTM	6.5%	–	(not retested)
GRU	5.2%	–	(not retested)

hint that the gap reported in Section 5 is mostly attributable to free-run drift, not to model capacity. A full sweep of α and the corresponding LSTM/GRU runs are left to future work.

4. Related Work

Having reported our own numbers, we can now position them precisely against the prior art. We organise the discussion around the same benchmark used in this paper, so that every external figure is directly comparable to Tables 2–4. A side-by-side summary is given in Table 5.

4.1. Polynomial system identification on this benchmark

The benchmark originates with Vlaar *et al.* [1], who fit *subject-specific second-order Volterra* series to the same wrist-perturbation/EEG pair. Their model captures the dominant nonlinear component of the cortical response and reports a free-run VAF $\approx 46\%$, which is to date the most relevant external reference for the closed-loop regime we target. Gu *et al.* [4] replace the Volterra expansion with a *polynomial NARMAX* structure whose terms are selected by Forward Regression with Orthogonal Least Squares (FROLS), the same selection mechanism that we reuse in our NNARX-FROLS variant (Section 2.4). Their one-step-ahead accuracy is very high (VAF_{OSA} $\approx 94\%$) but degrades to VAF ≈ 47 – 55% already at the three-step horizon, illustrating the same accuracy collapse with horizon length that we observe across all families. A precursor of that work [3] stacks a small *hierarchical neural network* on top of NARMAX regressors and pushes the three-step VAF to $\approx 69\%$.

4.2. Ensemble and evolutionary methods

More recently, Santos *et al.* [5] approach the same decoding problem with a *stacking ensemble* whose base learners are tuned by adaptive differential evolution (JADE), re-

porting VAF_{OSA} $\approx 94.5\%$ and a three-step VAF $\approx 67.5\%$. These methods reach the highest published accuracy on the benchmark, but at the cost of (i) subject-specific fitting and (ii) a markedly higher modelling and tuning complexity than the compact, shared-protocol neural models considered here.

4.3. Neural identification in the broader literature

Outside this benchmark, deep learning for system identification has matured along two lines: convolutional/temporal architectures that learn the regressor implicitly [14], and differentiable block-oriented models such as dynoNet [15] that embed linear dynamical blocks inside an end-to-end trainable network. A recurring lesson of this literature—and the one most relevant to our findings—is that the *training objective*, not the raw model capacity, governs closed-loop behaviour: models trained on one-step loss drift in free-run unless explicitly exposed to their own predictions, through scheduled sampling [13], multi-step losses, or state-noise injection. Our k -step curriculum (Section 2.5) is precisely such a mechanism, and the gap between our NNARX (curriculum trained) and recurrent (one-step trained) families is a direct, controlled demonstration of this principle on biomedical data.

4.4. Positioning of this work

Two features distinguish the published results from ours. *First*, all of [1, 3, 4, 5] fit *subject-specific* models, whereas our headline numbers use a Leave-One-Subject-Out protocol in which the test subject is entirely unseen; the literature figures are therefore an upper bound that a subject-agnostic model is not expected to reach. *Second*, the prior art reports accuracy almost exclusively in the one-step or low-order k -step regime, while we additionally characterise the full free-run simulation—the regime in which the model receives no online measurement of y . Read in this light, our blind free-run VAF $\approx 27\%$ and our Kalman-assisted VAF $\approx 58\%$ (Table 4) bracket the Volterra baseline of [1] from below and from above, locating the 46% figure squarely inside the interval spanned by the amount of sensor feedback available at inference time.

5. Discussion

5.1. Why does NNARX outperform the recurrent models in free run?

The headline finding—a plain MLP over a fixed regressor beats every recurrent cell in closed-loop simulation—is, at first sight, counter-intuitive. There are three concurrent reasons.

First, the NNARX is trained on a k -step curriculum up to $k = 320$ samples and is therefore explicitly exposed to its own predictions during training. The recurrent models,

Table 5. Positioning against the published literature on the same wrist-perturbation/EEG benchmark. “Subj.-spec.” indicates a subject-specific fit; “LOSO” the leave-one-subject-out protocol of this work. Dashes mark regimes not reported by the corresponding study. Literature figures are reproduced from the cited papers and are not re-run here; our figures are from Tables 2 and 4.

Method	Protocol	VAF _{OSA}	VAF _{3-step}	VAF _{free-run}	Reference
Volterra (2nd order)	subj.-spec.	–	–	$\approx 46\%$	Vlaar <i>et al.</i> [1]
NARMAX (FROLS)	subj.-spec.	$\approx 94\%$	47–55%	–	Gu <i>et al.</i> [4]
NARMAX + hierarchical NN	subj.-spec.	–	$\approx 69\%$	–	Gu <i>et al.</i> [3]
Stacking ensemble (JADE)	subj.-spec.	$\approx 94.5\%$	$\approx 67.5\%$	–	Santos <i>et al.</i> [5]
NNARX-FROLS (blind)	LOSO	84.7%	28.6%	27.2%	this work
NNARX-FROLS ($\alpha = 0.15$)	LOSO	–	–	58.5%	this work

even though they ingest the input sequentially, are trained on a one-step MSE that does not penalise long-horizon drift; scheduled sampling [13] and noise injection in the hidden state are known mitigations but were not used here, to keep the protocol uniform across families.

Second, the dataset is small: ten subjects times seven realisations of 30 s each, after a $\times 8$ downsampling, amount to a few minutes of effective training data. The NNARX benefits from a very strong inductive bias (it explicitly receives both $u[k-i]$ and $y[k-j]$ as inputs); the recurrent models must learn to construct an equivalent regressor inside their hidden state.

Third, the recurrent training pipeline is sensitive to back-propagation-through-time truncation, gradient clipping and warm-up length. We used standard, fixed values for all three; tuning them on a per-subject basis is a natural next step.

5.2. Gap with the literature

Our best free-run VAF (27%) is roughly 20 percentage points below the Volterra baseline of [1] (46%) and substantially worse than the subject-specific NARMAX of [4]. The main explanation is that Vlaar and Gu both fit *subject-specific* models, while we use a LOSO protocol that treats the test subject as completely unseen; the literature numbers are therefore an upper bound that the present setting is not allowed to match. Subject-specific fine-tuning of the NNARX-FROLS model is the most natural way to close the residual gap and is left to future work.

On one-step prediction the gap is much smaller: our VAF_{OSA} $\approx 85\%$ is within 10 percentage points of the 94% reported by the best NARMAX and ensemble methods, despite the much harder LOSO protocol.

5.3. Limitations

The study has several limitations worth stating explicitly. The hyper-parameter search is coarse: only a handful of widths and depths are explored per family. The recurrent training does not use scheduled sampling, hidden-state noise injection, or per-subject fine-tuning. Finally, al-

though we report wall-clock training time, energy and memory footprint are not measured.

6. Conclusion

We have compared four families of neural models for the identification of the cortical response to wrist-joint perturbations, on the publicly available Vlaar 2018 benchmark, using a unified preprocessing pipeline and a single configuration-driven notebook. On one-step ahead prediction every family is competitive ($\text{VAF} \geq 75\%$); in closed-loop free-run simulation only the NNARX family is usable, with the NNARX-FROLS variant peaking at $\text{VAF} \approx 27\%$ on a LOSO split, while RNN, LSTM and GRU collapse below 7%.

The result is a clean, perhaps cautionary, message: on a small biomedical dataset, an MLP over an explicitly designed regressor, trained with a k -step curriculum, beats every recurrent architecture we tried. A complementary Kalman-assisted simulation experiment (Section 3.6) shows that the gap between the blind free-run regime and the Volterra/NARMAX literature is dominated by autoregressive drift rather than by model capacity: with as little as 15% innovation feedback, the NNARX-FROLS variant climbs to $\text{VAF} \approx 58\%$ and the RNN to $\text{VAF} \approx 35\%$. Closing the residual gap in pure free-run, and rescuing the recurrent models with scheduled sampling and stability-aware losses, are the two natural directions for follow-up work.

References

- [1] M. P. Vlaar, T. Solis-Escalante, A. C. Schouten, and F. C. T. van der Helm. Modeling the nonlinear cortical response in EEG evoked by wrist joint manipulation. *IEEE Trans. Neural Syst. Rehabil. Eng.*, 26(1):205–215, 2018. 1, 2, 3, 5, 7, 8, 9
- [2] M. P. Vlaar *et al.* Benchmark dataset: Cortical responses evoked by wrist joint manipulation. 4TU.Centre for Research Data, doi:10.4121/uuid:176d8f78-d9fd-491e-90e7-9370e249b701. 3
- [3] Y. Gu, T. Solis-Escalante, A. C. Schouten, and F. C. T. van der Helm. A novel approach for modeling neural responses to joint perturbations using the NARMAX method and a

- hierarchical neural network. *Frontiers Comput. Neurosci.*, 12:96, 2018. 2, 5, 8, 9
- [4] Y. Gu, T. Solis-Escalante, A. C. Schouten, and F. C. T. van der Helm. Nonlinear modeling of cortical responses to mechanical wrist perturbations using the NARMAX method. *IEEE Trans. Biomed. Eng.*, 68(3):948–958, 2021. 2, 3, 4, 5, 8, 9
- [5] G. Santos *et al.* Decoding electroencephalography signal response by stacking ensemble learning and adaptive differential evolution. *Sensors*, 23(16):7049, 2023. 2, 8, 9
- [6] L. Ljung. *System Identification: Theory for the User*. Prentice Hall, 2nd edition, 1999. 1
- [7] O. Nelles. *Nonlinear System Identification*. Springer, 2001. 1
- [8] J. Sjöberg *et al.* Nonlinear black-box modeling in system identification: a unified overview. *Automatica*, 31(12):1691–1724, 1995. 1
- [9] M. Nørgaard, O. Ravn, N. K. Poulsen, and L. K. Hansen. *Neural Networks for Modelling and Control of Dynamic Systems*. Springer, 2000.
- [10] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997. 4
- [11] K. Cho *et al.* Learning phrase representations using RNN encoder-decoder for statistical machine translation. In *Proc. EMNLP*, 2014. 4
- [12] J. L. Elman. Finding structure in time. *Cognitive Science*, 14(2):179–211, 1990. 4
- [13] S. Bengio, O. Vinyals, N. Jaitly, and N. Shazeer. Scheduled sampling for sequence prediction with recurrent neural networks. In *Proc. NeurIPS*, 2015. 2, 8, 9
- [14] C. Andersson, A. H. Ribeiro, K. Tiels, N. Wahlström, and T. B. Schön. Deep convolutional networks in system identification. In *Proc. IEEE CDC*, 2019. 1, 8
- [15] M. Forgione and D. Piga. dynoNet: A neural network architecture for learning dynamical systems. *Int. J. Adapt. Control Signal Process.*, 35(4):612–626, 2021. 1, 8
- [16] E. Brookner. *Tracking and Kalman Filtering Made Easy*. Wiley, 1998. 4